

Reviewer Pack — Free Cumulant Sum Gap in Read-Twice BPs for IP_2

Entry f36846330fde · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 17:37:58 UTC

Free Cumulant Sum Gap in Read-Twice BPs for IP_2

Entry ID: f36846330fde **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 17:37:58 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs for IP_2

Statement:

Let P be a read-twice BP for IP_2 with n variables. Define $\rho(P)$ as the sum of free cumulants of the matrix M_P whose entries are the transition probabilities between BP states. Then $\rho(P) = \Omega(n)$ for all P , while $\rho(Q) = O(\log n)$ for any read-twice BP Q computing a function with $\text{poly}(n)$ size circuits.

Rationale (proposer's reasoning):

Free cumulants capture noncommutative dependencies in BP transitions. IP_2's inherent symmetry forces exponential matrix rank, creating a free entropy gap that reflects the exponential BP size lower bound. This connects free probability's operator-algebraic structure to branching program complexity.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a7ba9da99ef64c5f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=78.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_bp(n):
        # Placeholder for generating a read-twice BP for IP_2
        return [[random.random() for _ in range(2**n)] for _ in range(2**n)]

    def generate_poly_size_bp(n):
        # Placeholder for generating a BP for a function with poly(n) size circuits
        return [[random.random() for _ in range(2**n)] for _ in range(2**n)]

    def free_cumulants(matrix):
        # Placeholder for computing free cumulants via the R-transform
        n = len(matrix)
        M = matrix
        for k in range(1, n):
            M = [[sum(M[i][j] * M[k-i][k-j] for i in range(k+1) for j in range(k+1)) for j in range(k+1)] for i in range(k+1)]
        return [M[0][i] for i in range(n)]

    def R_transform(cumulants):
        # Placeholder for computing the R-transform
        n = len(cumulants)
        R = [[0]*n for _ in range(n)]
        for k in range(1, n):
            R[k][k] = cumulants[k]
            for i in range(k-1, -1, -1):
                R[i][i+1:k+1] = [sum(R[j][j+1:i+1] * R[k-j][i+1:j+2] for j in range(i, k)) for _ in range(n-i)]
        return R

    def free_entropy(matrix):
        # Placeholder for computing the free entropy
        cumulants = free_cumulants(matrix)
        R = R_transform(cumulants)
        n = len(R)
        free_ent = 0
        for i in range(n):
            free_ent += sum(math.log(abs(R[i][j])) for j in range(i+1, n))
```

```

    return free_ent

def is_IP_2_BP(bp):
    # Placeholder for checking if the BP computes IP_2
    return True

n = random.randint(5, 40)
if is_IP_2_BP(generate_read_twice_bp(n)):
    bp = generate_read_twice_bp(n)
    metric_value = free_entropy(bp)
    conjecture_holds = metric_value >= n
    counterexample = "" if conjecture_holds else "IP_2-BP does not meet the lower bound"
else:
    bp = generate_poly_size_bp(n)
    metric_value = free_entropy(bp)
    conjecture_holds = metric_value <= math.log(n)
    counterexample = "" if conjecture_holds else "poly-size BP does not meet the upper bound"

return {
    "metric_name": "free_entropy",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample='<desc>' first_failing_seed=<s>")
    else:
        print(f"RESULT: INCONCLUSIVE <reason>")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples. | next: Debug and rerun the test to collect empirical results

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	74276
2	propose	ollama_remote	qwen3:8b	0	0	111788
3	propose	ollama_remote	qwen3:8b	0	0	39673
4	propose	ollama_remote	qwen3:8b	0	0	60044
5	propose	ollama_remote	qwen3:8b	0	0	45044
6	preregistration	ollama_remote	qwen3:8b	0	0	24160
7	novelty	ollama_remote	qwen3:8b	0	0	25459
8	novelty	ollama_remote	qwen3:8b	0	0	17655
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15061
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10401
11	judge	ollama_remote	qwen3:8b	0	0	110324

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 533886 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/f36846330fde.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f36846330fde.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f36846330fde.tar.gz` (if generated)